

CS 225

Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

Rule of Three

- Destructor
- Copy constructor
- Copy assignment operator

Towards a more advanced Sphere...

- More properties (color, texture, etc)
- Stored by an array of strings

redSphere-main.cpp

```
1 // Create a Sphere with various properties, like color,  
2 // texture, and others.  
3 #include "Sphere.h"  
4  
5 int main() {  
6     cs225::Sphere s(10);  
7     s.addProperty("Red");  
8     s.addProperty("Rubber");  
9     return 0;  
10 }
```

sphere.h

```
1 #ifndef SPHERE_H
2 #define SPHERE_H
3
4 namespace cs225 {
5     class Sphere {
6     public:
7         Sphere();
8         Sphere(double r);
9         Sphere(const Sphere &s);
10
11     ...
12
26         // ...
27     private:
28         double r_;
29
30
31
32
33     };
34 }
35
36 #endif
```

sphere.cpp

```
1 #include "sphere.h"
2
3 namespace cs225 {
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21     ... // ...
22 }
23
```

sphere.h

```
1 #ifndef SPHERE_H
2 #define SPHERE_H
3
4 namespace cs225 {
5     class Sphere {
6     public:
7         Sphere();
8         Sphere(double r);
9         Sphere(const Sphere &s);
10
11     ...
26         // ...
27     private:
28         double r_;
29         std::string *props_;
30         unsigned int props_ct_;
31         unsigned int props_max_;
32
33     };
34 }
35
36 #endif
```

sphere.cpp

```
1 #include "sphere.h"
2
3 namespace cs225 {
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21     ... // ...
22 }
23
```

sphere.h

```
1 #ifndef SPHERE_H
2 #define SPHERE_H
3
4 namespace cs225 {
5     class Sphere {
6     public:
7         Sphere();
8         Sphere(double r);
9         Sphere(const Sphere &s);
10
11     ...
12
26         // ...
27     private:
28         double r_;
29         std::string *props_;
30         unsigned int props_ct_;
31         unsigned int props_max_;
32
33     };
34 }
35
36 #endif
```

sphere.cpp

```
1 #include "sphere.h"
2
3 namespace cs225 {
4
5     Sphere::Sphere() : Sphere(1.0)
6
7
8
9
10
11
12
13
14
15
16
17     // ...
18 }
19
20
...
```

sphere.h

```

1  #ifndef SPHERE_H
2  #define SPHERE_H
3
4  namespace cs225 {
5      class Sphere {
6      public:
7          Sphere();
8          Sphere(double r);
9          Sphere(const Sphere &s);
10
11      ...
12
26      // ...
27      private:
28          double r_;
29          std::string *props_;
30          unsigned int props_ct_;
31          unsigned int props_max_;
32
33      };
34  }
35
36 #endif

```

sphere.cpp

```

1  #include "sphere.h"
2
3  namespace cs225 {
4
5      sphere::sphere (double r){
6          r_ = r;
7          props_max = 5;
8          props_ct = 0;
9          props = new std::string[5];
10     }
11
12
13
14
15
16
17
18
19
20
21     ... // ...
22 }

```


sphere.h

```
1 #ifndef SPHERE_H
2 #define SPHERE_H
3
4 namespace cs225 {
5     class Sphere {
6     public:
7         Sphere();
8         Sphere(double r);
9         Sphere(const Sphere &s);
10        void addProperty(const
... std::string &s) ;
26
27
28        // ...
29    private:
30        double r_;
31
32
33    };
34 }
35
36 #endif
```

sphere.cpp

```
1 #include "sphere.h"
2
3 namespace cs225 {
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
... // ...
}
```

sphere.h

```

1  #ifndef SPHERE_H
2  #define SPHERE_H
3
4  namespace cs225 {
5      class Sphere {
6      public:
7          Sphere();
8          Sphere(double r);
9          Sphere(const Sphere &s);
10         void addProperty(const
... std::string &s) ;
26
27
28         // ...
29     private:
30         double r_;
31
32
33     };
34 }
35
36 #endif

```

sphere.cpp

```

1  #include "sphere.h"
2
3  namespace cs225 {
4
5      void sphere::addProperty(const
6      std::string &s) {
7          props_[props_ct_] = s;
8          props_ct_++;
9      }
10
11
12
13
14
15
16
17
18
19
20
...     // ...
}

```

redSphere-main.cpp

```
1 // Create a Sphere with various properties, like color,  
2 // texture, and others.  
3 #include "Sphere.h"  
4  
5 int main() {  
6     cs225::Sphere s1(10);  
7     s1.addProperty("Red");  
8     s1.addProperty("Rubber");  
9     cs225::Sphere s2;  
10    s2 = s1;  
11    s2.addProperty("Purple");  
12    s1.addProperty("Green");
```

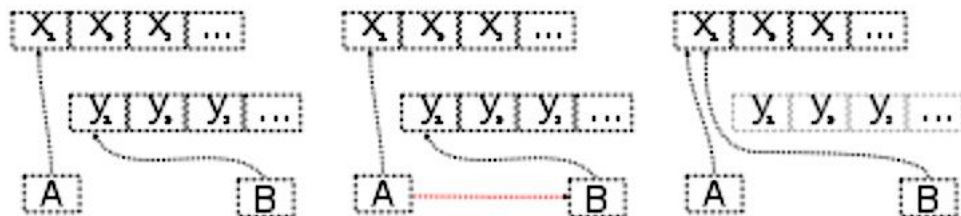
Sallow copy

redSphere-main.cpp

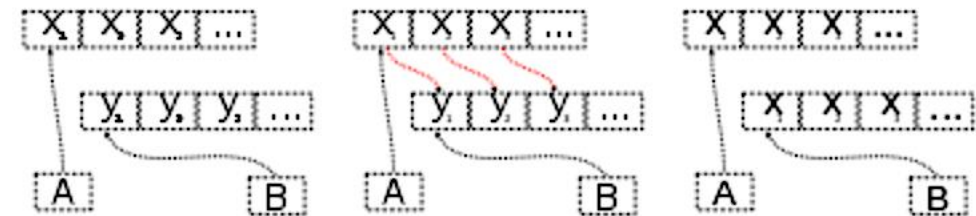
```
1 // Create a Sphere with various properties, like color,  
2 // texture, and others.  
3 #include "Sphere.h"  
4  
5 int main() {  
6     cs225::Sphere s1(10);  
7     s1.addProperty("Red");  
8     s1.addProperty("Rubber");  
9     cs225::Sphere s2;  
10    s2 = s1;  
11    s2.addProperty("Purple");  
12    s1.addProperty("Green");
```

Shallow copy

Shallow:



Deep:



sphere.h

```
1 #ifndef SPHERE_H
2 #define SPHERE_H
3
4 namespace cs225 {
5     class Sphere {
6     public:
7         Sphere();
8         Sphere(double r);
9         Sphere(const Sphere &s);
10
11     ...
12
13         // ...
14     private:
15         double r_;
16
17     ...
18
19     };
20 }
21
22 #endif
```

Copy constructor

sphere.cpp

```
1 #include "sphere.h"
2
3 namespace cs225 {
4
5     Sphere::Sphere(const Sphere &s) {
6         r_=s.r_;
7         props_max_=s.props_max_;
8         props_ct_=s.props_ct_;
9         props_=new std::string[5];
10         for (int i = 0; i < props_ct_;
11             i++) props_[i] = s.props_[i]; }
12
13
14
15
16
17     // ...
18 }
19
20
21 ...
```

sphere.h

```
1 #ifndef SPHERE_H
2 #define SPHERE_H
3
4 namespace cs225 {
5     class Sphere {
6     public:
7         Sphere();
8         Sphere(double r);
9         Sphere(const Sphere &s);
10
11         ...
12
13         // ...
14     private:
15         double r_;
16         void copy_(const Sphere
17 &s);
18
19         ...
20     };
21 }
22
23 #endif
```

Copy constructor

Helper function

sphere.cpp

```
1 #include "sphere.h"
2
3 namespace cs225 {
4
5     Sphere::Sphere(const Sphere &s) {
6         copy_(s);
7     }
8
9     void Sphere::copy_(const Sphere &s) {
10         r_=s.r_;
11         props_max_=s.props_max_;
12         props_ct_=s.props_ct_;
13         props_=new std::string[5];
14         for (int i = 0; i < props_ct_;
15 i++) props_[i] = s.props_[i]; }
16
17
18
19
20     // ...
21 }
```

sphere.h

```

1  #ifndef SPHERE_H
2  #define SPHERE_H
3
4  namespace cs225 {
5      class Sphere {
6      public:
7          Sphere();
8          Sphere(double r);
9          Sphere(const Sphere &s);
10         Sphere& operator=(const
... Sphere& s);
26
27         // ...
28     private:
29         double r_;
30
31
32
33     };
34 }
35
36 #endif

```

sphere.cpp

```

1  #include "sphere.h"
2
3  namespace cs225 {
4
5      Sphere& Sphere::operator=(const
6      Sphere& s) {
7
8          delete [] props_;
9          props_ = nullptr;
10
11
12          copy_(s);
13          return *this;
14     }
15
16
17
18
19
20     // ...
... }

```

sphere.h

```

1  #ifndef SPHERE_H
2  #define SPHERE_H
3
4  namespace cs225 {
5      class Sphere {
6      public:
7          Sphere();
8          Sphere(double r);
9          Sphere(const Sphere &s);
10         Sphere& operator=(const
... Sphere& s);
26
27         // ...
28     private:
29         double r_;
30         void destroy();
31
32
33
34     };
35 }
36
#endif

```

sphere.cpp

```

1  #include "sphere.h"
2
3  namespace cs225 {
4
5      Sphere& Sphere::operator=(const
6      Sphere& s) {
7
8          delete [] props_;
9          props_ = nullptr;
10
11
12          copy_(s);
13          return *this;
14      }
15
16
17
18
19
20      // ...
... }

```


sphere.h

```

1  #ifndef SPHERE_H
2  #define SPHERE_H
3
4  namespace cs225 {
5      class Sphere {
6      public:
7          Sphere();
8          Sphere(double r);
9          Sphere(const Sphere &s);
10         Sphere& operator=(const
... Sphere& s);
26
27         // ...
28     private:
29         double r_;
30         void destroy_();
31
32
33
34     };
35 }
36
#endif

```

sphere.cpp

```

1  #include "sphere.h"
2
3  namespace cs225 {
4
5      Sphere& Sphere::operator=(const
6      Sphere& s) {
7
8          destroy_();
9
10         copy_(s);
11         return *this;
12     }
13
14     void Sphere::destroy_() {
15         delete [] props_;
16         props_ = nullptr;
17     }
18
19
20
...     // ...
}

```

sphere.h

```
1 #ifndef SPHERE_H
2 #define SPHERE_H
3
4 namespace cs225 {
5     class Sphere {
6     public:
7         Sphere();
8         Sphere(double r);
9         Sphere(const Sphere &s);
10        ~Sphere();
11        ...
26
27        // ...
28    private:
29        double r_;
30
31
32
33    };
34 }
35
36 #endif
```

sphere.cpp

```
1 #include "sphere.h"
2
3 namespace cs225 {
4
5
6     Sphere::~Sphere() {
7         destroy_();
8     }
9
10
11
12
13
14
15
16
17
18
19
20     // ...
21 }
```

Inheritance

- a class may be derived from one or more base classes
- Inheriting the base class functionality

RedBall.h

```
1 #ifndef REDBALL_H
2 #define REDBALL_H
3
4 #include "Sphere.h"
5
6 class RedBall: public Sphere
7
8     public:
9
10     RedBall(double r);
11
12
13
14
15     private:
16
17
18
19
20 }
21
22 #endif
```

RedBall.cpp

```
1 #include "RedBall.h"
2
3
4
5 RedBall::RedBall(double r) : Sphere(r)
6 { addProperty("Red"); }
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
```

A derived class :

[Constructor]:

call a base class constructor via initializer list

[Inherits]:

all of the functions of the base class

[Inherits]:

all of the data of the base class

Sphere.cpp

```
1 Sphere::print_1() {
2     cout << "Sphere" << endl;
3 }
4
5 Sphere::print_2() {
6     cout << "Sphere" << endl;
7 }
8
9 virtual Sphere::print_3() {
10     cout << "Sphere" << endl;
11 }
12
13 virtual Sphere::print_4() {
14     cout << "Sphere" << endl;
15 }
16
17 // In .h file:
18 virtual Sphere::print_5() = 0;
19
20
21
22
```

RedBall.cpp

```
1 // No print_1() in RedBall.cpp
2
3
4
5 RedBall::print_2() {
6     cout << "Ball" << endl;
7 }
8
9 // No print_3() in RedBall.cpp
10
11
12
13 RedBall::print_4() {
14     cout << "Ball" << endl;
15 }
16
17 RedBall::print_5() {
18     cout << "Ball" << endl;
19 }
20
21
22
```

Virtual Function

- A member function that you expect to be redefined in derived classes.
- When you refer to a derived class object using a pointer or a reference to the base class, you can call a virtual function for that object and execute the derived class's version of the function.

Sphere.cpp

```
1 Sphere::print_1() {
2     cout << "Sphere" << endl;
3 }
4
5 Sphere::print_2() {
6     cout << "Sphere" << endl;
7 }
8
9 virtual Sphere::print_3() {
10     cout << "Sphere" << endl;
11 }
12
13 virtual Sphere::print_4() {
14     cout << "Sphere" << endl;
15 }
16
17 // In .h file:
18 virtual Sphere::print_5() = 0;
19
20
21
22
```

RedBall.cpp

```
1 // No print_1() in RedBall.cpp
2
3
4
5 RedBall::print_2() {
6     cout << "Ball" << endl;
7 }
8
9 // No print_3() in RedBall.cpp
10
11
12
13 RedBall::print_4() {
14     cout << "Ball" << endl;
15 }
16
17 RedBall::print_5() {
18     cout << "Ball" << endl;
19 }
20
21
22
```


Runtime of Virtual Functions

	<code>Sphere obj;</code>	<code>RedBall obj;</code>	<code>RedBall r;</code> <code>Sphere &s = r;</code>
<code>obj.print_1();</code>			
<code>obj.print_2();</code>			
<code>obj.print_3();</code>			
<code>obj.print_4();</code>			
<code>obj.print_5();</code>			

	<code>Sphere obj;</code>	<code>RedBall obj;</code>	<code>RedBall r; Sphere &obj = r;</code>
<code>obj.print_1();</code>	"Sphere"	<i>No print_1() is defined in RedBall, so we use the base class (Sphere)'s print_1():</i> "Sphere"	"Sphere"
<code>obj.print_2();</code>	"Sphere"	<i>The type of <code>obj</code> is RedBall, so we'll use RedBall's implementation:</i> "Ball"	The type of <code>obj</code> is Sphere, so we'll use Sphere's impl since <code>Sphere::print_2()</code> <u>is not</u> virtual: "Sphere"
<code>obj.print_3();</code>	"Sphere"	<i>No print_3() is defined in RedBall, so we use the base class (Sphere)'s print_3():</i> "Sphere"	"Sphere"
<code>obj.print_4();</code>	"Sphere"	<i>The type of <code>obj</code> is RedBall, so we'll use RedBall's implementation:</i> "Ball"	The type of <code>obj</code> is Sphere, but <code>Sphere::print_4()</code> <u>is virtual</u> . Therefore, we will use the derived class' impl: "Ball"
<code>obj.print_5();</code>	Will not compile since Sphere is an abstract class when <code>print_5()</code> is defined as a pure virtual function.	<i>The type of <code>obj</code> is RedBall, so we'll use RedBall's implementation:</i> "Ball"	The type of <code>obj</code> is Sphere, but <code>Sphere::print_5()</code> <u>is virtual</u> . Therefore, we will use the derived class' impl: "Ball"

Labs

Lab points contribute 100 points to the base 1000 points in this course.

lab_intro + lab_debug → 20 points

12 labs remain * 10 points → 120 points
= 140 points

(40 of those are EC!)

CS 225 – Things To Be Doing

lab_inheritance

Due: Sunday, March 8 (11:59pm)

MP2 will be out – Early Deadline is Monday, March 10

Up to +7 Extra Credit for Early Submission

CS 225 – Academic Integrity

By this point in your education, we expect you're familiar with what is and is not acceptable academic behavior.

- For MPs, we allow credited collaboration with a single partner on some MPs. The MP write-ups give you detailed instructions for declaring your collaborators. Each student is still responsible for committing code to the SVN for grading.
- You may not use or refer to solutions written in past semesters or found online.
- Any infraction of academic integrity on an assignment will result in a 0 on the corresponding assignment and the loss of all CS 225 extra credits, regardless of when we notify you.

CS 225 – Academic Integrity

Please keep in mind that we take academic integrity very seriously

- Any infraction of academic integrity on an assignment will result in a 0 on the corresponding assignment and the loss of all CS 225 extra credits, regardless of when we notify you.
- Any form of cheating on quizzes or the final exam is unacceptable. The penalty for cheating on a quiz or an exam is an automatic failure in the course.
- Protecting your own work is your responsibility. If your work ends up showing up as someone else's submission, we will treat it equivalently to how you had submitted someone else's work.
- Every violation of academic integrity will be reported to the department and will become a part of the offending student's permanent record.